X03｜怎样证明重放没有多建一张工单？

本章只围绕一项承诺选证据：**同一 key 与同一输入重放，取回原工单，不增加已经完成的业务效果。** response、数据库、外部计数和日志各补一段，不能互相替代。

第二次传送，怎样仍然指向第一件事？

当前问题	关系或短用法	记到什么程度
哪个值要稳定，哪个值应换新？	创建 key 与业务 payload 保持，request ID 区分第二次处理；201→200，replayed 从 false→true。	能同时表达“一件事”和“两次传送”；它们都不是用户身份或权限。
同一 key 携带不同主题会怎样？	本例对照规范化 payload 摘要，不同返回 409，原行不变。	**重放不等于借旧 key 修改原意图。** 先判断是修改工单还是新业务动作。
所有重放都不能增加外部 calls 吗？	已收束 confirmed/manual_review 直接读回；pending 可能继续交接，calls 增加但 effect 不应重复。	根据先前状态选预期，不把正常重放的计数当全局规则。

回看：「两次请求怎样指向同一件事？」。

“没有重复”需要哪几份互补证据?

当前问题	关系或短用法	记到什么程度
两次 response 都是 T-42 就够吗?	比较 201→200、replayed 标记以及全部公开工单字段，之后 GET 还应一致。	证明公开合同；仍可能藏有重复数据，需要数据库核对。
本地查一张票，还要看创建事件吗?	正常重放预期 tickets=1、ticket_created=1；防止重复持久化或记录来源不一致。	数据与事件共同核对，不只看返回的编号。
provider calls/effects 解决哪个疑问?	正常已收束重放都维持 1；pending 恢复可为 2/1。	本地一张票不保证远端只有一项安排；计数含义依场景。
第二次日志能补什么?	正常重放应有 ticket_intent_replayed，不再走新的 commit/partner 事件链。	说明控制路径；没有日志仍不能单独证明没有效果。
查到一行，怎样确认是这次应用的数据?	核对数据库文件/连接目标和环境；旧副本的查询结果不说明当前 API。	**查询对象身份与查询内容同样重要。**

直接查物理列时，认熟 `status AS ticket_status`：前者是本例数据库列，后者只是结果别名；没有新增或重命名表中字段。按 `WHERE ticket_id = :ticket_id` 绑定目标编号，完整 SELECT 回看「SQL 查询为什么还要关心物理字段名?」。

组合断言程序 `x03_replay_check.py` 回看「用一段可运行实验把这些断言接在一起」；它使用临时真实 SQLite 与有状态 Fake，核对票、事件、调用效果、日志和 GET。断言取决于场景，改 payload 后应转成 409 的检查，不能沿用成功重放的预期。证据分工回看「四份证据怎样互相补足?」。

哪一种检查能回答当前问题，又没经过哪里?

当前问题	关系或短用法	记到什么程度
只想查为什么没发请求，用整套 UAT 吗?	纯函数测试固定 `"3"`、`"urgent"`、`"-3"` 的转换与范围，保留被查实现。	按当前承诺选小检查，不让最终 201 断言盖住更早原因。
要查事务有没有一起撤回?	在 event INSERT 处受控失败，检查票、事件与计数器；不只断言抛了异常。	围绕要保护的状态选择证据。
要查提交后丢回复是否重复生效?	partner 测试保留稳定 key 与内容，模拟丢回复，检查 calls=2/effects=1。	Fake 验证受控边界的本地处理，不证明真实供应商合同。
TestClient 通过，端口就通了吗?	它在进程内进入 API；外部 HTTPX 可被 MockTransport 接住，不经过真实 listener。	**接口行为测试与网络运行证据分开。**
双进程主机检查比它增加什么?	`host_integration_check.py` 启动 API 与模拟合作方两个进程，经 loopback 真正传 JSON，分别用 tickets.db 与 receipts.db，并查重启持久性。	增加 socket、端口、进程证据；仍未经过 Compose DNS、镜像或 named volume。
受控 AI 测试说明什么?	固定坏引用、越权建议，核对拒绝、工具次数与工单不变。	证明这套授权和候选检查行为，不证明真实模型的普遍质量。

`pytest` 入口与小测试分工回看「为什么还要保留小测试?」；主机检查回看「从进程内协作扩大到两个真实进程」。后者建立并清理自己的临时进程与目录，仍是真实运行动作；只读教材不等于这些检查已执行。

一次 UAT 和它的日志，要怎样读才不扩大结论?

当前问题	关系或短用法	记到什么程度
X 的一次 UAT 包含哪些交接?	创建、同意图重放、GET 读回、政策草稿，四次请求分别取 ID。	与 F 的 live/ready/create/read 序列分清；按本例实际入口查。
fresh 与 replay 模式在确认什么?	`--initial-create-mode fresh` 要求首条新建；replay 要求首条已是旧意图；模式错就失败。	模式表达起始事实，不是为了让输出变绿任选。
切换 AI 场景还要再造工单吗?	已完成工单用 replay 起步；close_proposal 应 needs_human、tool_permission_denied、tool_called=false、票仍 open。	独立政策请求不改变创建基线；配置恢复也不撤销旧工单。
`uat.py` 知道合作方内部效果有几项吗?	它核公开工单、政策状态/原因/来源/工具标记；内部调用效果另由 Fake 记录或专门检查取得。	**公开 UAT 不能凭空观察远端内部状态。**
正常日志与重放日志为何不同?	正常含两次事务与外部尝试；收束后的重放主要记录 intent_replayed；政策有自己的候选检查事件。	用阶段链解释当前请求，不要求所有请求走同一链。
日志说已提交，数据库却没有，信谁?	核对日志发生时机与数据库目标；日志文字不能覆盖持久事实。	保留不一致并定位，别挑一份绿色证据宣布成功。

case_id 是“哪种条件”，request_id 是“哪次 HTTP 处理”，同一用例可发多次请求。普通日志只保留允许的 ID、阶段、结果、版本和计数，不存 subject、prompt、候选正文、token 或原始异常。回看：「一次 UAT 为什么有四个请求编号?」「日志怎样解释一次处理，又怎样保护输入?」。